Human Operator Decision Support for Highly Transient Industrial Processes: A Reinforcement Learning Approach

JIANQI RUAN, Purdue University, USA

BOB NOONING, IVAN PARKES, and WAL BLEJDE, Castrip LLC, USA

GEORGE T.-C. CHIU and NEERA JAIN, Purdue University, USA

Most industrial processes are not fully-automated. Although fast and low-level control can be handled by controllers, initializing and adjusting the reference, or setpoint, values, are commonly tasks assigned to human operators. A major challenge is the variation in skill and experience among different operators which leads to variation in how they determine operation setpoints. In turn this can result in inconsistencies in the final product. A typical approach to automating the determination of operational setpoints might be to bypass the operators' experience and directly learn the optimal control strategy by allowing the agent to interact with the process. However, given that to train an agent on a real industrial process could pose unacceptable process safety or stability risks, we propose a modified actor-critic algorithm to learn purely from human operators' recorded behavior without any requirements for interaction between the agent and the process to be controlled. By redesigning the advantage function, we show that the agent can independently control the process by selecting the most rewardable action that has been observed in the operators' past behavior under the same operating conditions. The agent's performance is verified using casting data from an industrial twin-roll steel strip casting process.

CCS Concepts: • **Human-centered computing** → *Human computer interaction (HCI)*; • **Computing methodologies** → **Machine learning**; • **Theory of computation** → **Theory and algorithms for application domains**.

Additional Key Words and Phrases: Reinforcement Learning, Human-in-the-loop, Decision aid, Twin-roll strip casting

ACM Reference Format:

Jianqi Ruan, Bob Nooning, Ivan Parkes, Wal Blejde, George T.-C. Chiu, and Neera Jain. 2022. Human Operator Decision Support for Highly Transient Industrial Processes: A Reinforcement Learning Approach. In . ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

In most industrial processes, although relatively fast time-scale and low-level control tasks are automated, human operators still manually adjust process setpoints. The manual adjustments frequently occur during the start-up operations of industrial processes because they are typically highly transient and difficult to model or control. In [14], such a human-in-the-loop control scenario is defined as a supervisory control scenario, in which human operators supervise the controller by monitoring the system feedback and adjusting setpoints accordingly. However, factors like personality traits [20], past experiences [9], skill level [13], and even the current mood of an operator[8], can cause variations in human strategy and lead to inconsistencies in process operation [15]. Therefore, there remains a challenging, albeit fascinating, problem of how to improve the consistency and performance of human operators in executing start-up operations.

Manuscript submitted to ACM

Table 1. Nomenclature

Symbol	**Description**
S	State
A	Action
$\Delta(.)$	Difference in (.) between two consecutive steps
D	Training database
F	Roll separation force setpoint value
R	Immediate reward
N	Number of samples in the training database
Φ	Parameter set of the value function
Ψ	Parameter set of the policy function
Sub/superscript	**Description**
$(.)_k$	Discrete time index
$(.)_i$	Sample index in a database
$(.)_{lb}$	Lower bound of (.)

Researchers have employed different modeling techniques to estimate human operation behavior [1, 4, 12] and subsequently design a supervisory controller using the identified behavior model. Certainly, automating the supervisory role of the human operator would likely improve consistency in start-up operations. However, identifying a human behavior model becomes nontrivial when the dataset for identifying the human model contains multiple distinguishable human behaviors; should the model follow one of these human behaviors, or should it blend multiple different behaviors? Moreover, control strategies developed by human operators might have their own strengths and weakness when considering different operational scenarios. In other words, even if a particular human behavior can be distinguished from the dataset and modeled precisely, the subsequent model-based controller will not necessarily improve the closed-loop system performance.

On the other hand, researchers have also investigated determining the optimal operator input to the system with a process model constructed using either data-driven methods or first principles. In [10], Hu et al. propose an optimal performance reference model with an optimal control approach for a power system. In [22], Vasconcelos et al. construct a simulated environment allowing operators to explore operation strategies for chemical processes. In [23], Wei et al. propose a reinforcement learning (RL) algorithm for learning the optimal operation strategies from a system simulation of a cutter section dredger. Indeed, if a first-principles model of the given system is available, model-based control approaches can work well. However, if the system model is established purely based on a training dataset from human operation records, then nonintuitive state-input pairs might rarely or never appear in the training dataset, and thus the model precision under such pairs could be poor and subsequently affect the optimal control policy developed from such a model.

In our previous work [17], we proposed a modified deep Q-network (DQN) algorithm to train a decision support agent using a finite dataset from human operators' records. Because there is no system model or simulator available for free exploration, and allowing a training agent to explore the new policy on a real plant may result in unacceptable process safety or stability risks which cannot be tolerated, we offset the immediate reward with a positive number to force the agent to choose the most rewardable action among actions selected by operators under a similar situation. In the work, we showed that the trained agent can independently adjust one setpoint based on a single objective. However, as the state vector grows, as well as the complexity of the overall reward function (e.g. considering multiple time-varying objectives), this approach does not scale well. A second challenge with this approach is that it cannot overcome the negative consequences of an imbalanced dataset in which some human actions are overrepresented as compared to others.

In this paper, we propose a modified actor-critic algorithm to train a decision support agent for operation setpoint determination for industrial processes characterized by complex dynamics and multiple performance objectives. Instead of applying an offset to the reward function, we take the advantage of the actor-critic algorithm, which trains the policy function as a multiple-class classification problem, so that cost-sensitive methods can be applied to update the policy function based on both the reward and the action distribution in the dataset. In addition, we apply this method to learn a setpoint control strategy in a twin-roll strip casting process and show that the trained agent can independently make reasonable and consistent setpoint adjustments under a given scenario.

This paper is organized as follows. In Section 2 we describe the proposed modified actor-critic algorithm. In Section 3 we provide an example of applying the algorithm to a twin-roll steel strip manufacturing process, and in Section 4 we compare the performance between the trained agent and the human operator. In Section 5, conclusions and future work are discussed.

2 ALGORITHM

In the actor-critic algorithm [3], separate algorithms are used to estimate the value function and the policy function. The value (critic) function V maps a state to its value, which is defined as the expected long term reward starting from the given state; that is $V : S_k \rightarrow \mathbb{E}\left(\sum_{t=k}^{\infty} \gamma^{t-k} R_t | S_k\right)$. The policy (actor) function π maps a state-action pair to a probability value between 0 and 1, which represents, under this policy, how likely the action A_k is to be taken at the given state S_k. Considering a finite training dataset, the value function can be evaluated as shown in Algorithm 1.

Algorithm 1 Pseudocode of the value function training process

1: Initialize γ
2: Form the training dataset with samples: $d_i \in D, d_i = \{S_{ki}, A_{ki}, S_{(k+1)i}, R_{ki}\}, i = 1, 2, ..., N$
3: $\Phi_0 = \text{argmin} \sum_{d_i \in D} |V(S_{ki}|\Phi) - R_{ki}|_2$
4: **for** f = 1:iteration **do**
5:     **for** $d_i \in D$ **do**
6:         Calculate $\bar{v}_i = R_{ki} + \gamma(1 - \beta_i)V(S_{(k+1)i}|\Phi_{f-1})$
        (* β_i is a binary indicator, indicating if the state S_{ki} is the end state of a sequence.)
7:     **end for**
8:     $\Phi_f = \text{argmin} \sum_{d_i \in D} |V(S_{ki}|\Phi) - \bar{v}_i|_2$
9: **end for**

If new observations are collected, one can always add them to the dataset D and increase training iterations. However, since we consider a finite training set, the converged value function V will be fixed and used for training the policy function. The training process of the policy function involves updating the likelihood of choosing a certain action under the given state according to an advantage value α citepmlr-v48-mniha16. As shown in Equation (1), if the sum of the immediate reward R_{ki} and the discounted value of the subsequent state $\gamma V(S_{(k+1)i})$ is greater than the value of the current state $V(S_{ki})$, then the action A_{ki} is considered a valuable one, and its likelihood given S_{ki} should be increased based on how much the advantage is. However, if the advantage value α_i is negative, the updated policy function is less likely to choose A_{ki} when encountering S_{ki}. When free exploration in a real or simulated environment is not allowable, a negative advantage value may increase the likelihood of the policy selecting an action not represented in the dataset. In other words, the consequence of that action in terms of the resultant state is unknown. To mitigate this issue, we use e^{α_i} to determine how much to increase the likelihood of $\pi(A_{ki}|S_{ki})$. Since e^{α_i} is always positive, a less valuable action observed in the dataset will still have a higher chance of being selected compared to those actions that have never been taken, given a certain state.

In addition, the finite training dataset might have an uneven distribution in terms of the actions taken by the human operators. To effectively learn from an imbalanced dataset, researchers have developed methods such as re-sampling [2, 6], random forest [7], and cost-sensitive methods [19]. Re-sampling is not a challenge when free exploration is available since the agent can interact with the environment and up-sample those actions which are less common. However, when free exploration is not possible, the cost-sensitive method is an effective methodology to implement in the policy function update scenario. We define $\eta(A_{ki})$ to be the likelihood of action A_{ki} appearing within the training dataset D. The loss function depends on both the $\eta(A_{ki})$ and the e^{α_i}. As shown in Equation (2), if an action is frequently taken in the training dataset and has little or negative advantage value, its weight will be low in the loss function. The training process of the policy function is shown in Algorithm 2.

Algorithm 2 Pseudocode of the policy function training process

Initialize learning rate a, discount factor γ
Form the training dataset with samples: $d_i \in D, d_i = \{S_{ki}, A_{ki}, S_{(k+1)i}, R_{ki}\}, i = 1, 2, ..., N$
Input Φ^*, the parameters of the value function from Algorithm 1
Randomly initialize Ψ_0, the parameter set of the policy function
for f = 1:iteration **do**
    $Loss = 0$
    for $d_i \in D$ **do**
        Calculate the advantage value

$$\alpha_i = R_{ki} + \gamma(1 - \beta_i)V(S_{(k+1)i}|\Phi^*) - V(S_{ki}|\Phi^*) \tag{1}$$

        (* β_i is a binary indicator, indicating if state S_{ki} is the end state of a sequence.)
        Update the loss

$$Loss = Loss - \frac{e^{\alpha_i}}{\eta(A_{ki})} \log(\pi(A_{ki}|S_{ki}, \Psi_{f-1})) \tag{2}$$

    end for
    $\Psi_f = \Psi_{f-1} - a\nabla_\Psi Loss$
end for

3 TWIN-ROLL CASTING EXAMPLE

To demonstrate the performance of the proposed algorithm, we apply it to twin-roll steel strip casting. Here we describe this manufacturing process and the application of the algorithm to it.

3.1 Twin-Roll Steel Strip Casting

Twin-roll steel strip casting is a near-net-shape manufacturing process used to improve energy efficiency and reduce operating costs for steel manufacturing [5]. Different from traditional thick-slab casting, molten steel is poured directly onto the surfaces of two casting rolls, cooled, and formed into a thin strip in the twin-roll casting process. The process is difficult to model and control due to its rapid thermo-mechanical dynamics, and this difficulty increases during the highly transient start-up process. Thus, human operators are employed to manually adjust certain setpoints to 1) achieve the desired characteristics of the final product by the end of the start-up process, and 2) maintain these characteristics through the entire steady-state operating process.

In this study, we mainly focus on the setpoint adjustments during the start-up process. A cast sequence start-up process is defined as the period from when the casting process begins to the start of producing the first prime coil.

Due to factors such as steel temperature and heat flux, cast sequences even with the same grade of steel and same final product specifications can have varying initial characteristics related to final product quality. Operators are asked to drive the system operation to a steady state that satisfies several performance specifications by making a series of binary decisions (switches) and adjusting multiple setpoints. Over twenty switches and setpoints can be adjusted by operators; for the setpoints, operators not only need to decide when to adjust each setpoint but also by how much. Due to the problem complexity, this work focuses on one specific setpoint of interest during the start-up process.

3.2 Force Setpoint Adjustment

During the start-up process, the casting roll separation force setpoint (to be referred to as the "force setpoint") is the most frequently adjusted setpoint. Operators adjust the force setpoint to respond to different strip profile issues. In our previous work [17], we described how the strip chatter (C), a non-negative value indicating the thickness variation along the cast length direction [24], is a major factor driving the force setpoint adjustment. In addition, operators might adjust the force setpoint to respond to another category of profile imperfections, edge spikes. Unlike chatter, which describes profile imperfections along the cast length direction, edge spikes are profile imperfections that lie along the strip cross section. Four parameters are used to characterize different edge spike problems:

(1) Edge bulge (bg): among 0 to 25% edge region from the outer end, the thickness range from the peak to the closest minima in the direction away from the outer end. It is a non-negative value.
(2) Edge ridge (eg): among 25% to 50% edge region from the outer end, the thickness range from the peak to the closest minima in the direction away from the outer end. It is a non-negative value.
(3) Maximum peak (mp): maximum thickness between the edge bulge and edge ridge locations with respect to the inner end of the edge region. It is a real value.
(4) High edge flag (fg): a binary value indicating whether either edge region is thicker than the cross section center thickness. Figure 2 shows a scenario where edge region is thicker than the center region.

Generally, increasing the force setpoint increases the force applied on the strip surface and reduces the amount of the semi-solid material (also known as "mushy" material) between the solidified shells, which mitigates some edge spike problems. However, the mushy material functions as a damper, which reduces strip vibration. Therefore, the reduction of the mushy material results in less damping and more vibration in the strip which in turn worsens the chatter problem. In other words, there is a trade-off between mitigating chatter versus mitigating edge spike problems.

3.3 Algorithm Settings

3.3.1 State and Action. Given that modeling the system dynamics during the start-up process can be difficult, the reinforcement learning agent considered here is designed to learn by only observing human operation data collected a priori and then suggest the optimal setpoint adjustment (value and timing) to the human operator. The state at time step k is composed of

$$S_k = \{C_k, \Delta C_k, F_k, \Delta F_k, bg_k, \Delta bg_k, eg_k, \Delta eg_k, fg_k, \Delta fg_k, mp_k, \Delta mp_k\}, \tag{3}$$

where $\Delta(.) = (.)_k - (.)_{k-1}$ is the difference between values of the current and previous time steps. Cast data is recorded at 1 Hz and smoothed with a 10-second moving-average filter [21]. In addition, observing that human operators do not adjust the force setpoint more frequently than 0.2 Hz, we further downsample the data at 0.2 Hz to adapt to the force setpoint adjustment frequency used by human operators.

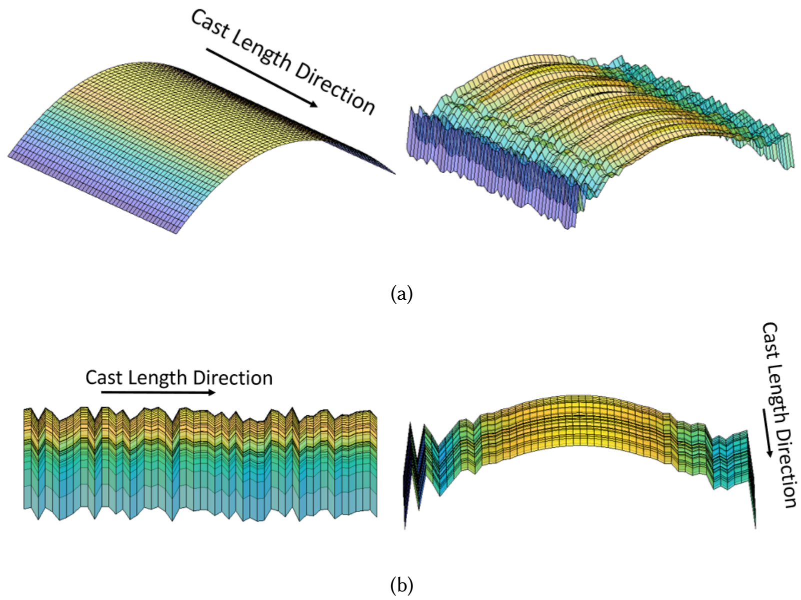


Fig. 1. Notional images of different strip characteristics. a) Left: an ideal strip profile has a parabolic shape on its cross section and no thickness variation along the strip length direction. Right: A strip affected by both chatter and edge spike problems ripples both along the length direction and in each cross section. b) Left: thickness variation along the strip length affected by chatter. Right: ripples in each cross section, especially problematic on the edges.

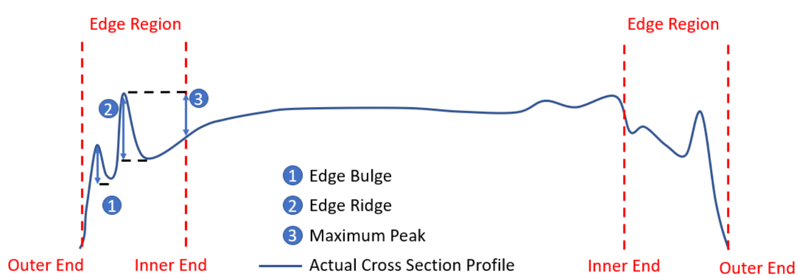


Fig. 2. Schematic of a notional strip cross section describing the edge spike parameters.

It has also been observed that operators typically adjust the force setpoint by one of eight fixed values. Therefore, at a time step k, the agent is allowed to adjust the force setpoint by one of eight values: $A_k \in \{a_j, j = 1, 2, ..., 8\}$. Among these actions, three represent decreasing the force setpoint, four represent increasing the force setpoint, and one is defined as keeping the force setpoint unchanged. A challenging aspect of the specific problem under consideration is that when human operators keep the force setpoint constant, it is not known whether that action was taken deliberately, or if it represents more passive behavior that resulted from an operator being distracted by other operation tasks. In the next section we will describe how we address this ambiguity in our algorithm.

3.3.2 Reward Function. The reward function explicitly incentivizes desired performance metrics. In this study, edge spike and chatter are the profile characteristics that we want to mitigate with force setpoint adjustments during a start-up process. The chatter problem is characterized by the chatter parameter value, and the edge spike is characterized by the edge bulge, edge ridge, and maximum peak parameters. The high edge flag parameter is not used to characterize the edge spike problem because it is a binary value and is not comparable to the other three parameters related to edge spike. However, the high edge flag information is embedded in the state vector to provide the agent with extra information to make a decision. It is desirable to have low values of chatter, edge bulge, edge ridge, maximum peak, and a decreasing trend of these parameters. However, once the value of a parameter decreases below a user-defined threshold, continuing to decrease its value is not necessary. Based on these observations, we define an edge spike parameter $P_k = \max(bg_k, eg_k, mp_k)$ and construct a piecewise defined reward function for the performance objectives as

$$R_{S_k} = \begin{cases} -\frac{\Delta P_k}{2W_{\Delta P}} - \frac{\Delta C_k}{2W_{\Delta C}} \text{ if } P_k < P_{lb} \text{ and } C_k < C_{lb} \\ -\frac{\Delta P_k}{W_{\Delta P}} + \frac{(P_{lb}-P_k)}{W_P} - \frac{\Delta C_k}{2W_{\Delta C}} \text{ if } P_k \geq P_{lb} \text{ and } C_k < C_{lb} \\ -\frac{\Delta P_k}{2W_{\Delta P}} - \frac{\Delta C_k}{W_{\Delta C}} + \frac{(C_{lb}-C_k)}{W_C} \text{ if } P_k < P_{lb} \text{ and } C_k \geq C_{lb} \\ -\frac{\Delta P_k}{W_{\Delta P}} + \frac{(P_{lb}-P_k)}{W_P} - \frac{\Delta C_k}{W_{\Delta C}} + \frac{(C_{lb}-C_k)}{W_C} \text{ if } P_k \geq P_{lb} \text{ and } C_k \geq C_{lb} \end{cases}, \tag{4}$$

where $W_{\Delta(.)}$ is the weight used to scale $\Delta(.)$ in the range $[-1, 1]$, $W_{(.)}$ is the weight used to scale $(.)$ in the range $[-2, 2]$, and C_{lb} and P_{lb} are user-defined thresholds for the chatter and edge spike parameters.

3.3.3 Imbalanced Dataset. To categorize different force setpoint adjustment behaviors, we employ a k-means clustering algorithm [11, 18] to cluster 95 individual cast sequences in the training dataset. The start-up process of each sequence is operated by one of six human operators. All of the cast sequences represent the same steel grade and strip width and are collected from the same cast machine to avoid behavior variation caused by differences in the cast conditions.

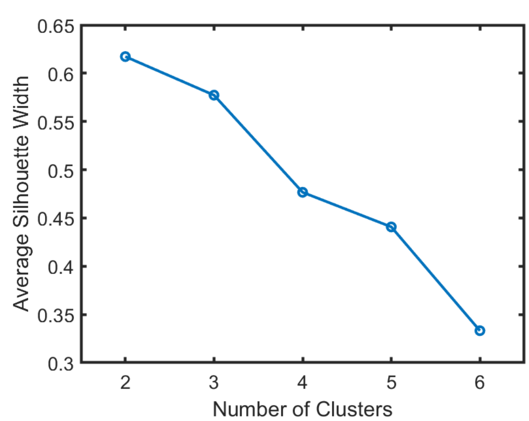
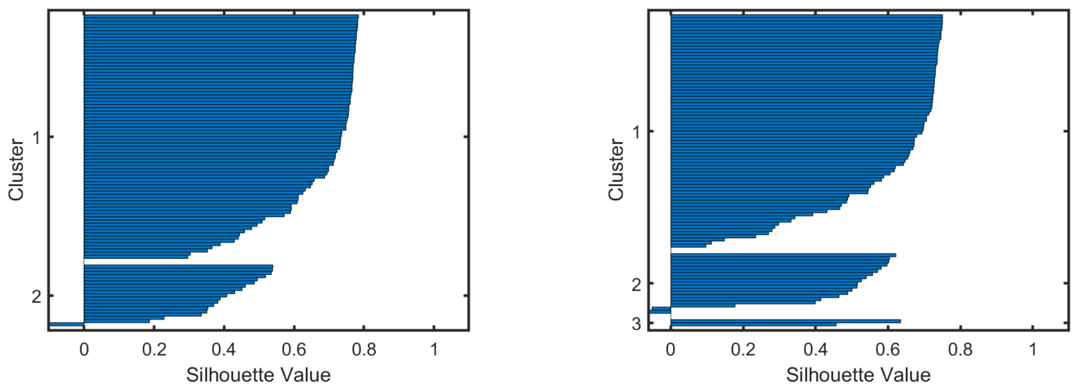


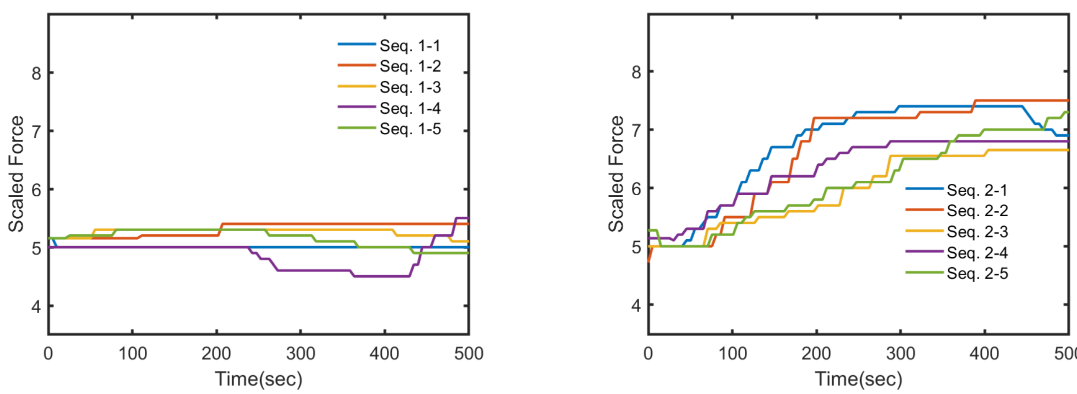
Fig. 3. Average silhouette width of $k = \{2, 3, ..., 6\}$: the clustering result of $k = 2$ has the highest average silhouette width

The force setpoint adjustment behavior is characterized by a 500-second force setpoint trajectory after the manual mode of the force setpoint begins. Since there are 6 operators, we evaluate the clustering results of $k = \{2, 3, ..., 6\}$. The average silhouette width [16] indicates that both $k = 2$ and $k = 3$ have an average silhouette width higher than 0.5. According to Figure 4, there is no major difference between $k = 2$ and $k = 3$. Therefore, for simplicity, we use the clustering results of $k = 2$. Figure 4a also shows an uneven distribution in the clustering. Combined with the force trajectory examples shown in Figure 5, we observe that over 70% of the sequences belong to Cluster 1, which is characterized by less aggressive behavior in both force adjustment range and frequency. In addition, over 90% of the samples in the training dataset have the zero-force-change action, $a_4 = 0$. In Section 4 we will show that the proposed algorithm can overcome the challenges posed by these imbalances in the training dataset.



(a) $k = 2$ (b) $k = 3$

Fig. 4. Silhouette width of each sample under different clustering settings; from $k = 2$ to $k = 3$, there are only two sequences being clustered in the additional cluster.



(a) Cluster 1 (b) Cluster 2

Fig. 5. Examples of force trajectories in different clusters; Cluster 2 has a larger range of force setpoint values compared to Cluster 1.

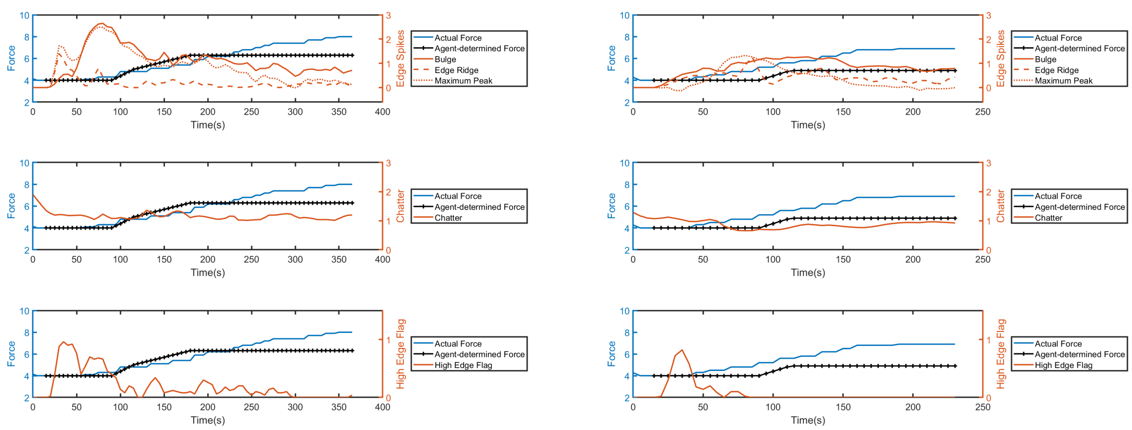
3.3.4 Neural Network Architecture. Both the value function and the policy function are represented as neural networks. The selection of the neural network architectures is heuristic and shown in Table 2. The value function has 701 learnable parameters, and the policy function has 848 learnable parameters. The total number of samples used to train these two neural networks is $N = 4594$.

Table 2. Neural network architectures of the value function and the policy function

value function	**policy function**
fully connected layer (12 $\longrightarrow$ 20)	fully connected layer (12 $\longrightarrow$ 20)
tanh activation layer	leaky ReLU activation layer
fully connected layer (20 $\longrightarrow$ 20)	fully connected layer (20 $\longrightarrow$ 20)
tanh activation layer	leaky ReLU activation layer
fully connected output layer (20 $\longrightarrow$ 1)	fully connected output layer (20 $\longrightarrow$ 8)
	softmax activation layer

4 RESULTS

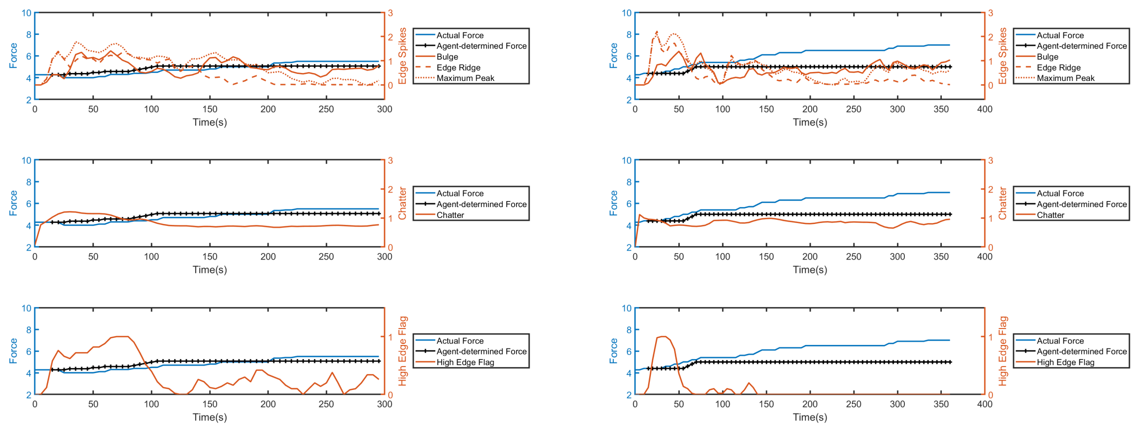
In the testing process, 95 cast sequences are used for training the reinforcement learning agent, and another 8 cast sequences with the same metal grade and width condition are used for testing. Except for the force setpoint values chosen by the human operator $F, \Delta F$, the other defined states are provided to the agent at each time step. At the initial time step, the agent observes the initial force setpoint value and is required to adjust it based on the state information; the decision made by the agent affects the subsequent step's force setpoint value. The goal of this test is to verify whether the trained agent reacts to the twin-roll casting process in a manner that is intuitive given the presence of a particular imperfection in the steel strip. Figures 6 and 7 show two pairs of testing sequence comparisons. The action force (blue curve) represents the human operator's actual force trajectory, and the force prediction (black "+" curve) represents the agent's force trajectory.



(a) Case 1 (b) Case 2

Fig. 6. Reinforcement learning agent verification under conditions exhibiting different initial edge spikes. Case 1 exhibits higher edge spike values compared to Case 2. Correspondingly, the agent increases the force setpoint value more aggressively in Case 1. However, the human operators' trajectories in both cases are similar to each other.

These comparisons demonstrate two important points. The first point is demonstrated in Figure 6. When the process is behaving differently between two casts, the agent makes different setpoint decisions; this is desired and expected, and is a contrast to the human operator behavior that was observed in these two casts. In these casts, the underlying human trajectories were similar despite the differences in how the process was behaving. The second point is demonstrated in



(a) Case 1 (b) Case 2

Fig. 7. Reinforcement learning agent verification under an operating condition exhibiting similar initial edge spike. Both cases exhibit relatively low instances of imperfections due to chatter and edge spikes. In both cases, the agent performs consistently and rarely increases the force setpoint value. However, the human operator in Case 2 increases the force setpoint value more aggressive compared to the operator in Case 1.

Figure 7. When the objective-related parameters are similar between two casts, the agent likewise makes consistent decisions in the two casts. This is in contrast to what the human operator did in the actual casts, which was to make different force setpoint value decisions despite the process behaving similarly.

Although these results do not represent closed-loop interaction between the agent and the twin-roll casting process, they provide valuable insight into how the agent would be behave under different casting scenarios. They also demonstrate how the proposed algorithm is mitigating the challenges of an imbalanced dataset. For example, the results show that the agent can aggressively increase the force when the underlying edge spike values are high, rather than keep the force constant, which is the action most frequently observed in the training dataset. Moreover, the trajectory similarity between the human operator and the RL agent is observed in both Figures 6 and 7's Case 1, which is a consequence of forcing the agent to choose actions that resemble what the human has done in the past by using the modified advantage value.

5 CONCLUSION

In this work, we proposed a modified actor-critic algorithm to design a decision support aid for human operators in highly transient industrial process applications. By assigning different update weights to the likelihood policy function based on the data distribution and an advantage value, the proposed algorithm can train a reinforcement learning agent with an imbalanced set of observations. We applied the proposed algorithm to a twin-roll steel strip manufacturing process to learn a force setpoint control strategy from human operators data and select the most feasible action that 1) has been taken under a similar situation in human records, 2) is most rewardable among all actions taken under similar situations. Ninety-five individual cast sequences were used to train the agent, and eight unseen cast sequences were used to test. The testing (verification) required the agent to decide the force setpoint adjustment at each time step based on a recorded state vector. Verification of the agent using existing cast data showed that the agent can improve consistency in setpoint decision-making as compared to that of human operators, and likewise that it can produce more intuitive decisions than those demonstrated by human operators. Future work is needed to validate the agent on the actual industrial casting process, as well as additional training using larger datasets with more variation in human operator behavior.

6 ACKNOWLEDGMENTS

We would like to thank all engineers and operators at Nucor Corporation who provide us with experimental data as well as invaluable technical knowledge of twin-roll casting.

REFERENCES

[1] Ozkan Celik and Seniz Ertugrul. 2010. Predictive human operator model to be utilized as a controller using linear, neuro-fuzzy and fuzzy-ARX modeling techniques. *Engineering Applications of Artificial Intelligence* 23, 4 (2010), 595–603.

[2] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. 2002. SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research* 16 (2002), 321–357.

[3] Thomas Degris, Martha White, and Richard S. Sutton. 2012. Off-Policy Actor-Critic. *CoRR* abs/1205.4839 (2012). arXiv:1205.4839 http://arxiv.org/abs/1205.4839

[4] YM Enab. 1995. Controller design for an unknown process, using simulation of a human operator. *Engineering Applications of Artificial Intelligence* 8, 3 (1995), 299–308.

[5] RIL Guthrie and MM Isac. 2016. Conventional and near net shape casting options for steel sheet. *Ironmaking & Steelmaking* 43, 9 (2016), 650–658.

[6] Hui Han, Wen-Yuan Wang, and Bing-Huan Mao. 2005. Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning. In *Advances in Intelligent Computing*, De-Shuang Huang, Xiao-Ping Zhang, and Guang-Bin Huang (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 878–887.

[7] Haibo He and Edwardo A. Garcia. 2009. Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering* 21, 9 (2009), 1263–1284. https://doi.org/10.1109/TKDE.2008.239

[8] Glyn Robert John Hockey. 1986. Changes in operator efficiency as a function of environmental stress, fatigue, and circadian rhythms. *Handbook of perception and human performance* 2. Cognitive processes and performance (1986), 1–49.

[9] Robert R Hoffman, Matthew Johnson, Jeffrey M Bradshaw, and Al Underbrink. 2013. Trust in automation. *IEEE Intelligent Systems* 28, 1 (2013), 84–88.

[10] Wan-Lin Hu, Claudio Rivetta, Erin MacDonald, and David P Chassin. 2019. Optimal operator training reference models for human-in-the-loop systems. In *Proceedings of the 52nd Hawaii International Conference on System Sciences*.

[11] K. Krishna and M. Narasimha Murty. 1999. Genetic K-means algorithm. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 29, 3 (1999), 433–439. https://doi.org/10.1109/3477.764879

[12] Sirajum Munir, John A. Stankovic, Chieh-Jan Mike Liang, and Shan Lin. 2013. Cyber Physical System Challenges for Human-in-the-Loop Control. In *8th International Workshop on Feedback Computing (Feedback Computing 13)*. USENIX Association, San Jose, CA. https://www.usenix.org/conference/feedbackcomputing13/workshop-program/presentation/munir

[13] Michael C Nechyba and Yangsheng Xu. 1995. Human skill transfer: neural networks as learners and teachers. In *Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots*, Vol. 3. IEEE, 314–319.

[14] David Sousa Nunes, Pei Zhang, and Jorge Sá Silva. 2015. A survey on human-in-the-loop applications towards an internet of all. *IEEE Communications Surveys & Tutorials* 17, 2 (2015), 944–965.

[15] Jeffrey R Peters, Vaibhav Srivastava, Grant S Taylor, Amit Surana, Miguel P Eckstein, and Francesco Bullo. 2015. Human supervisory control of robotic teams: Integrating cognitive modeling with engineering design. *IEEE Control Systems Magazine* 35, 6 (2015), 57–80.

[16] Peter J. Rousseeuw. 1987. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *J. Comput. Appl. Math.* 20 (1987), 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

[17] Jianqi Ruan, Michael Ponder, Ivan Parkes, Wal Blejde, George Chiu, and Neera Jain. 2022. Applying a Deep Q-Network for Human Operator Behavioral Modeling and Decision Support in a Twin-Roll Casting Process. In *Proceedings of the 2022 American Control Conference*. 689–696.

[18] Shashi Sharma and Ram Lal Yadav. 2013. Comparative Study of Kmeans and Robust Clustering. *International Journal of Advanced Computer Research* 3, 3 (2013), 4.

[19] Wei Shen, Xinggang Wang, Yan Wang, Xiang Bai, and Zhijiang Zhang. 2015. DeepContour: A Deep Convolutional Feature Learned by Positive-Sharing Loss for Contour Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[20] James L Szalma and Grant S Taylor. 2011. Individual differences in response to automation: the five factor model of personality. *Journal of Experimental Psychology: Applied* 17, 2 (2011), 71.

[21] Inc. The MathWorks. 2021. 1-D digital filter - MATLAB filter. shorturl.at/jtBOP

[22] Claudia JG Vasconcelos, Rubens Maciel Filho, Renato Spandri, and Maria R Wolf-Maciel. 2005. Dynamic models towards operator and engineer training: Virtual environment. In *Computer Aided Chemical Engineering*. Vol. 20. Elsevier, 565–570.

[23] Changyun Wei, Fusheng Ni, and Xiujing Chen. 2019. Obtaining human experience for intelligent dredger control: a reinforcement learning approach. *Applied Sciences* 9, 9 (2019), 1769.

[24] Peter Andrew WOODBERRY, Nikolco Nikolovski, and Wai Yee Daniel Yuen. Jan. 8, 2019. Method of operation of twin roll strip caster to reduce chatter. shorturl.at/ajBCG